Selective Residual Computation: A Causal Test of Refinement, Interference, and Gating

Jorge Simão

Paper 1 research draft — E1 results, August 2026

Abstract

Behavioral scores underdetermine how a language model uses depth. This paper is the first experimental test of the Paper 0 dual-selection framework. We ask whether Transformer residual blocks refine an evolving solution, contribute complementary information, or sometimes damage a task-conditioned trajectory that later blocks repair. E1 measures causal block utility and repair in tiny decoder-only Transformers; E2 holds content fixed while changing distributed goal evidence; E3 compares matched baseline and gated residual models. A shared goal latent is secondary, and hard skipping is conditional on a robust E1–E3 signal. We additionally use a released, closely matched Qwen3-based baseline/headwise-gated pair as an external probe: its per-token, per-head sigmoid gate explicitly separates an ungated attention candidate from the effective residual update. Geometry is descriptive, ablation is functional, and the term *strong interference* is reserved for replicated negative utility with later repair. In the completed E1 run, nine of twelve task–layer cells were useful, one negative-utility cell occurred only on an unlearned task, and the preregistered strong-interference criterion was not met.

1 Scientific objective

Paper 0 defines the broader dual-selection framework. Paper 1 instantiates it in Transformer residual streams and asks whether residual blocks refine, complement, interfere, or can be modulated or skipped under task context. The scope here is experimental: loss, accuracy, exact match, and F1 remain primary behavioral outcomes, while internal quantities discriminate among mechanisms that could produce similar scores.

We therefore treat trained neural networks as experimental cognitive systems and ask:

1. What information becomes active?
2. What transformations become active?
3. How do representations evolve across token time and model depth?
4. When do successive residual updates refine, complement, or interfere with one another?
5. Can a distributed task/goal state predict which transformations will be useful?
6. Can task-dependent inhibition become real computation skipping?
7. Which strongest tiny-model findings survive a pretrained gated-attention comparison?

Backpropagation and token streams are held fixed. Biological analogy, local learning rules, and embodied tasks are outside this paper's evidential scope.

1.1 Questions and preregistered decision structure

The confirmatory sequence is deliberately asymmetric. E1 must establish a residual phenomenon before E3 is interpreted as a remedy, and E2 must establish task dependence before a goal-conditioned explanation is advanced. The primary hypotheses are:

1. **H1 (task-conditioned utility):** at least some blocks have utility $U_l(\tau)$ that changes across counterfactual goals with content held fixed;

2. **H2 (strong interference):** reliably negative utility, when present, co-occurs with later repair or compensation and replicates across seeds;

3. **H3 (learned modulation):** matched residual gates reduce activation of blocks independently classified by E1 as harmful or redundant more than random, static-route, or norm-based controls;

4. **H4 (dual selection):** in pretrained gated attention, gate values contain information about functional update consequence beyond attention concentration and update magnitude;

5. **H5 (complementarity):** useful high-gate updates need not have positive state–update cosine.

Thresholds for negligible utility, minimum effect size, bootstrap confidence level, and required seed replication are fixed in the experiment registry before final test inspection. Failure to reject a null is reported with uncertainty rather than converted into evidence for redundancy.

2 Mechanistic measurement framework

Let $x_{l,t} \in \mathbb{R}^d$ be the state entering a candidate transformation at layer l and token position t. Define the raw candidate update $\Delta_{l,t}$, computational-selection variable $g_{l,t}$ (or $g_{l,h,t}$), and effective update

$$\widetilde{\Delta}_{l,t} = g_{l,t} \odot \Delta_{l,t}, \qquad x_{l+1,t} = x_{l,t} + \widetilde{\Delta}_{l,t}.$$

For an ungated block, $g = 1$ by convention, but computational selection is claimed only through intervention. Pre-norm state, SDPA output, output-projection result, MLP output, and post-block state are named separately in artifacts and never compared as if they occupied the same location. The metric battery is deliberately broad, but each family must discriminate among mechanistic hypotheses rather than become an interpretability dashboard.

2.1 Memory activation

Reuse PRA-style measurements wherever the architecture exposes retrievable or materialized memory: active-to-available KV ratio; retrieved references, chunks, or gists; supporting-fact recall; routing precision/recall/rank; evidence coverage; materialization breadth; and task quality versus active-memory budget.

A central cross-line hypothesis is that memory utility need not be monotonic:

$$Q(k+1) < Q(k)$$

may occur when additional materialized memory introduces distraction or representational interference. Sparse activation may therefore be functionally superior rather than merely cheaper.

2.2 Attention concentration and dilution

Measure attention entropy, effective support

$$N_{\text{eff}} = \exp\left(-\sum_i \alpha_i \log \alpha_i \right),$$

top-k mass, evidence mass, distance profile, concentration indices, head similarity, and changes with prompt, context, or memory size. Attention describes interaction structure; it is not treated as a complete causal explanation.

Memory selection and computational selection are crossed into four empirical regimes: sharp-attention/low-gate, sharp-attention/high-gate, diffuse-attention/low-gate, and diffuse-attention/high-gate. Their frequencies and task consequences are reported by layer and head. H4 requires gate information not explained by attention statistics alone.

2.3 Residual refinement, complementarity, and interference

Use update norms and cosines, covariance, CKA/RSA, subspace angles, task-variable decodability, logit trajectories, and causal ablation.

Refinement. Successive updates improve a common evolving representation or response. Candidate evidence includes monotonic increase in correct-answer margin, increasing decodability of task-relevant variables, and positive causal utility of successive blocks.

Complementarity. A layer adds useful information without substantially undoing prior computation. Candidate signatures include low update alignment, increased representational rank or new decodable factors, and positive causal utility for both earlier and later transformations.

Interference. A transformation creates a state that later computation must repair, or its removal improves performance in a task-dependent way. For block F_l and task family τ define

$$U_l(\tau) = Q_\tau(\text{full}) - Q_\tau(\text{skip } l).$$

Then $U_l > 0$ is useful, $U_l \approx 0$ redundant, and $U_l < 0$ a candidate harmful transformation.

Strong evidence for interference requires three conditions:

1. statistically reliable negative task-conditioned utility $U_l(\tau) < 0$;

2. evidence that later computation reverses, compensates for, or repairs the affected trajectory;

3. replication across examples and seeds, not an isolated ablation artifact.

Geometry alone is never sufficient to label a transformation harmful.

Because block skipping can create off-manifold states, the confirmatory analysis includes graded attenuation and, where shape and scale permit, norm-matched replacement or counterfactual activation patching. Conclusions state the intervention used. Negative U_l from a single skip is an *interference candidate*, not strong interference.

2.4 Temporal and multiscale stability

Token position provides a temporal axis distinct from model depth. We characterize input embeddings, hidden states, heads, features, and learned subspaces over multiple windows W.

For each feature/subspace compute rolling mean, variance, skewness, kurtosis and their first/second differences; adjacent and separated chunk distribution distances; covariance and correlation-matrix drift; autocorrelation function (ACF) and ACF drift; lagged cross-correlation between features, heads, and layers; effective correlation length; CKA/RSA across temporal windows; principal-subspace and eigenspectrum drift; and layer×scale×token-position stability maps.

We explicitly search for *amplification–repair* events: a perturbation, task conflict, or nonstationarity grows at one layer and is reduced at a later layer. Multiple temporal spans are required because local windows can appear stationary while longer windows reveal topic, goal, or discourse transitions.

2.5 Head and layer organization

For any representation, attention, temporal, or causal similarity metric M, compare

$$\mathbb{E}[M(\text{heads within a layer})] \quad \text{with} \quad \mathbb{E}[M(\text{heads across layers})].$$

Add variance decomposition by layer/head/task, clustering, specialization, redundancy, causal ablation utility, and temporal stability. This tests whether functional organization is primarily layered, head-specific, task-specific, or distributed.

2.6 Conventional ML/DL and systems metrics

Retain loss/perplexity, accuracy, EM, precision, recall, F1, calibration, output entropy, logit margin, gradient norms and directions, weight norms, activation norms, update-to-weight ratios, training stability, FLOPs, realized latency, memory use, throughput, and active-block fraction. Internal metrics are scientifically useful only when connected to competence, robustness, or resource cost.

Soft gating is not counted as computation saving when the candidate transformation has already been evaluated. Active-FLOP and latency claims are restricted to hard routes that avoid execution; theoretical FLOPs and realized wall-clock speed are reported separately.

3 Datasets and task ecology

Dataset structure is part of the causal experiment. Paper 1 remains in NLP/token streams. Mazes, Sudoku, 2-D environments, and embodied agent tasks are deferred to a separate paper.

3.1 Synthetic natural-language counterfactual mixtures

The basic unit is a *counterfactual task family*. Construct content objects C_i that support multiple plausible intentions I_j:

$$(C_i, I_j) \to Y_{ij}.$$

The same content is reused for answering, explaining, extracting, classifying, critiquing, transforming, checking, planning, and action-selection-like textual responses.

Task identity must not collapse to a discrete `MODE` token. Use paraphrased and implicit instructions; goal evidence distributed across conversational context; audience, style, safety, resource, and output constraints; conflicting local affordances; controlled distance between goal-defining evidence and response-affording content; semantically continuous task families; distractors; and same-content/different-goal and same-goal/different-content counterfactuals.

Define a controllable *goal identifiability* variable measuring how strongly local lexical cues reveal the intended task. This allows direct tests of whether interference increases when the task must be inferred from distributed context. Ground-truth latent task factors are retained for probing and causal evaluation only.

3.2 PRA continuity datasets

Use WikiText-2, WikiText-103, HotpotQA, and QASPER where feasible. WikiText supports unconstrained token dynamics and temporal stability analysis; HotpotQA provides supporting-fact supervision for memory/evidence activation; QASPER provides long scientific-document QA. Reuse creates direct continuity with PRA routing, attention dilution, materialization, and long-context measurements.

4 Architectures and experimental priority

All architectures expose a common instrumentation record for embeddings, residual pre/post states, attention outputs/weights, MLP outputs, per-head outputs where feasible, logits, gradients, candidate/effective updates, and architecture-specific latent states. Model-specific hooks live behind an adapter. Native instrumented forwards must match untouched logits exactly when possible, or within a declared tolerance, before measurements are accepted.

4.1 Core discovery models

The mandatory first-cycle matrix is:

1. tiny decoder-only Transformer;
2. tiny Transformer + residual gates;
3. tiny Transformer + shared goal latent z;
4. tiny Transformer + z-conditioned residual gates.

Tiny models are the mechanism-discovery environment because they permit exhaustive block ablation, many seeds, dense metric capture, and controlled architectural intervention.

4.2 Selective pretrained transfer

The external comparison uses the released 1B baseline and headwise variants (model revision `aad415c45ec6`). Their full identifiers and revisions are pinned in the run config. The released configurations match on 28 layers, hidden width 2048, 16 query heads, 8 key/value heads, vocabulary size 152064, maximum position setting 32768, tokenizer family, and bfloat16 dtype. The headwise model adds one gate logit per query head to each attention query projection, so capacity is close but not parameter-identical. Shared release provenance does not prove

identical data order or optimization history; comparisons are therefore described as closely matched pretrained contrasts, not a single-factor randomized experiment.

Verified gate location. Inspection of official implementation revision `f4c2a5f6ffd6` shows that the query projection packs query features and gate logits. For headwise gating the logits are reshaped to

$$g_l \in [0,1]^{B\times T\times H\times 1}$$

after a sigmoid. They multiply the per-head SDPA output after the head axis is moved to $[B, T, H, d_h]$ and before flattening and the output projection. Thus we distinguish the per-head ungated SDPA candidate $a_{l,h,t}$, its effective form $g_{l,h,t}a_{l,h,t}$, the post-projection attention write, and the residual state after addition. This sequence follows the release and the gated-attention study [5].

Pretrained gate interventions. Native gates are compared with forced-open, forced-closed, per-head mean, token-shuffled, and optional thresholded gates. Shuffling preserves marginal gate distributions. Native instrumentation is parity-tested separately from interventions. Candidate reconstruction from an effective output is rejected when a gate is too small for stable division; no clipped reconstruction is silently analyzed.

4.3 Conditional comparators

TRM, mHC, Mixture-of-Depths, LayerSkip, or other third-party architectures are conditional rather than mandatory for the first cycle. Choose based on the observed phenomenon: mHC if residual interference dominates; MoD/LayerSkip if conditional compute dominates; TRM if iterative refinement dominates.

5 Novel experimental mechanisms

The mechanisms are modular interventions, not assumed contributions.

5.1 Goal-conditioned residual gating

Replace

$$h_{l+1} = h_l + F_l(h_l)$$

with

$$h_{l+1} = h_l + g_l(z)F_l(h_l).$$

Start with scalar/block gates for causal clarity. Soft gates test modulation. Thresholded gates test actual skipping:

$$g_l(z) < \epsilon \Rightarrow F_l \text{ is not executed.}$$

5.2 Shared distributed goal latent

Introduce

$$z_{l+1} = z_l + G_l(z_l, h_l).$$

The goal latent is not supervised as a one-hot task identifier. It is learned through the task objective; probes are measurement devices. A key criterion is stronger than task classification: z should predict future causal utility,

$$z \to \widehat{U_l(\tau)}.$$

If z only encodes task semantics but cannot predict downstream computational requirements, it should not be interpreted as a useful control state.

5.3 Required goal-state controls

Compare z against current ordinary residual state h_l; pooled residual summaries over earlier layers/tokens; matched-dimensional random vectors with matched first/second-order statistics; shuffled-goal counterfactuals; a gate-only model; and a z-only model. These controls distinguish a genuine control state from generic additional capacity or dynamic sparsity.

5.4 Combined functional top-down modulation

Use

$$h_{l+1} = h_l + g_l(z_l)F_l(h_l), \qquad z_{l+1} = z_l + G_l(z_l, h_l).$$

"Top-down" is functional rather than literal backward layer connectivity: globally integrated task state regulates more local transformations.

5.5 Sufficient activation objective

Use

$$\mathcal{L} = \mathcal{L}_{task} + \lambda \sum_l c_l \phi(g_l)$$

and sweep λ. The strong hypothesis is not merely quality preservation under reduced compute. It is the existence of $C_1 \subset C_2$ such that

$$\mathrm{Cost}(C_1) < \mathrm{Cost}(C_2), \qquad Q(C_1) > Q(C_2).$$

This is the computation-side analogue of selective memory materialization.

6 Analysis and statistical design

The unit of analysis is declared for every result. Tiny-model claims use at least five independently trained seeds unless a resource exception is recorded. Example-level effects are paired within seed; seed summaries, rather than tokens treated as independent replicates, establish replication. Pretrained deterministic inference varies examples, task family, context condition, and prompt form. Closely matched but separately trained checkpoints are not analyzed as paired parameter realizations.

Primary estimates include sample count, mean, median where skewed, standard deviation, example-bootstrap confidence intervals, seed-level sign consistency, and effect size. Gate–metric relations include Pearson and Spearman correlations, binned curves, layer/head-stratified estimates, and pooled models with layer/head/task controls. A pooled correlation alone cannot support H4. Final test families are held out from architecture and threshold selection.

The minimum pretrained plot set is: gate distributions by layer and head; gate versus alignment, relative magnitude, downstream utility, and attention entropy; candidate versus effective-update geometry; intervention effects; and layerwise baseline-versus-gated dynamics. Plots preserve layer/head/token structure before aggregation.

6.1 Reproducibility and artifact contract

Every run stores config, seed, repository commit, environment, package versions, device/dtype, model and dataset revisions, tokenizer, context length, batch size, exact hook locations, gate semantics, and intervention mode. Derived observations retain run/model/dataset/task/example/token/layer/head identifiers and candidate/effective norms, cosines, attention statistics, quality, utility, and repair score. Large tables use Parquet; paper summaries use CSV. Raw activations are retained only when required to reproduce a derived result.

7 Staged experimental protocol

7.1 Core E1: baseline residual map

Train tiny baseline models across seeds; capture the common metric battery; exhaustively skip individual blocks and selected groups; compute $U_l(\tau)$; identify useful, redundant, and candidate harmful blocks; search for later repair; and measure task decodability, logit trajectories, attention dilution, head/layer similarity, and temporal stability. E1 establishes whether there is an architectural pathology worth fixing.

7.2 Core E2: counterfactual goal manipulation

Hold C fixed while changing distributed intent I. Measure

$$\Delta h_l = h_l(C, I_1) - h_l(C, I_2),$$

block utility, attention, logits, temporal statistics, and goal identifiability. Test whether task-conditioned utility changes systematically as goal evidence becomes less locally obvious.

7.3 Core E3: gated residual stream

Train matched baseline and gated tiny models. Compare task quality, harmful-block activation, repair signatures, gate entropy, task dependence, active FLOPs, realized latency where possible, and distractor robustness. Compare with random skipping, static task routes, magnitude/norm heuristics, and matched-capacity ungated controls.

7.4 Secondary E4: shared goal latent

Train z without gates. Test semantic organization, task-switch tracking, and especially prediction of future $U_l(\tau)$. Compare to residual/pooling/random controls.

7.5 Secondary E5: combined modulation

Combine z and gates. Test whether gates suppress blocks independently identified by E1 as harmful or redundant rather than simply learning arbitrary sparse routes.

7.6 Conditional E6: hard skipping and sufficient activation

Only after soft-gating structure is understood, convert strong inhibition into real execution skipping. Sweep ϵ and λ. Report Pareto fronts over quality, active FLOPs, realized latency, and measured interference.

7.7 Conditional E7: architecture transfer

First, run the released Qwen3-style baseline/headwise-gated pair on language modeling, retrieval or QA, multi-hop reasoning where supported, and the controlled counterfactual set. Test H4–H5 using native gates and gate interventions. Report conventional task metrics and the dual-selection regimes alongside internal measurements. Only then transfer the strongest tiny-model signature to whichever additional architecture matches the dominant finding.

7.8 Conditional E8: PRA memory–computation interaction

Where PRA is available, factorially vary memory materialization and active computation:

$$Q = Q(k_{\text{memory}}, k_{\text{compute}}).$$

Test whether excess memory increases residual interference and whether goal modulation changes optimal retrieval breadth.

8 Results to date

8.1 E1: baseline residual map

We trained four-layer, width-64 causal residual decoders (150,758 parameters) on deterministic natural-language counterfactual families. Each content triple was paired with maximum, minimum, and modulo-10-sum goals expressed at three lexical-identifiability levels without a task-ID token. Content families were disjoint across 720 training, 180 validation, and 270 frozen test examples. Five independently initialized models used seeds 11, 22, 33, 44, and 55. Thresholds and the test split were fixed in `configs/e1_baseline.yaml` before test analysis.

Mean test accuracy was 0.612 (seed SD 0.039, range 0.570–0.663). The aggregate concealed a sharp task difference: maximum and minimum accuracy were 0.880 and 0.858, whereas modulo-10 sum accuracy was 0.098, effectively chance for the ten possible answer words. Accuracy did not vary monotonically with the lexical-identifiability

manipulation (high 0.613, medium 0.631, low 0.591), so E1 provides no evidence that the nominal identifiability scale alone controlled difficulty.

Table 1: E1 task-conditioned block classification. Confidence intervals bootstrap seed means; a cell is candidate harmful only when its interval lies below -0.002 and at least four of five seed means cross that threshold.

Task	Useful	Uncertain	Candidate harmful	Strong interference
Maximum	4	0	0	No
Minimum	4	0	0	No
Modulo-10 sum	1	2	1	No

The sole candidate-harmful cell was the final block on modulo-10 sum: mean probability utility was -0.0073 with a 95% seed-bootstrap interval $[-0.0153, -0.0020]$, and four seed means were below the fixed -0.002 threshold. This does *not* support strong interference. The affected task was not learned, the fifth seed was near zero, and no example met the preregistered 25% downstream repair criterion. Moreover, a final block has no later block that could repair its write. We therefore retain this as a negative-utility observation on an underfit task and do not interpret it as a replicated interference mechanism.

Across the learned maximum/minimum tasks, every block was positively useful. Mean task–block utility ranged from 0.095 to 0.437 at the limits of the seed-level intervals. The first-cycle E1 result consequently favors distributed refinement over a pervasive harmful-block account. Per-example records, example-bootstrap intervals within seed, seed summaries, manifests, histories, and checkpoints are stored under `results/e1`. E2 will test the narrower surviving claim that utility changes with goal while content is fixed; it is not licensed to treat negative cosine or the isolated sum result as interference.

9 Primary claims, decision rules, and falsification

The paper starts with three nested claims.

Primary mechanistic claim. Residual blocks have task-dependent causal roles that can include refinement, complementarity, redundancy, and possibly harmful interference.

Secondary control claim. Distributed task/goal state predicts some variation in downstream block utility.

Stretch architectural claim. Goal-conditioned modulation exploits that structure to reduce interference or active computation and can improve quality or cost.

After E1–E4, choose the paper center by explicit decision rule:

- strong harmful/repaired block effects but weak gating $\Rightarrow$ interference characterization;
- strong goal-conditioned utility prediction $\Rightarrow$ distributed task-control state;
- strong modulation with quality gains $\Rightarrow$ goal-modulated residual computation;
- quality improves while compute falls $\Rightarrow$ sufficient/selective activation;
- weak/null effects $\Rightarrow$ comparative residual dynamics/falsification, with unsupported architecture claims removed.

Negative results are part of the design. If harmful blocks are rare, the interference hypothesis is weakened. If z does not beat ordinary residual summaries for predicting utility, the shared-control-state hypothesis is weakened. If gating only tracks update magnitude, it is interpreted as amplitude control. If attention statistics explain gates almost completely, the dual-selection claim is weakened. If forced-open gates barely change performance, native gate values are not described as causally selecting useful computation. If gating saves compute without changing interference, it is positioned as conditional computation rather than cognitive control.

10 Positioning and scope limits

This is a mechanism study, not a claim that Transformer gates instantiate biological inhibition. Representation similarity such as CKA [1], adaptive computation [2], dynamic depth such as Mixture-of-Depths [3], and LayerSkip [4] provide neighboring tools or comparators. The gated-attention release studies the performance, sparsity, stability, attention-sink, and long-context effects of post-SDPA gates [5]; the present contribution is to use its learned gate as an observable and intervenable variable in an architecture-independent residual-dynamics test. We do not attribute our cognitive interpretation to that work.

The present design cannot fully remove training-history confounds in pretrained comparisons, and block ablations may create off-distribution states. It addresses these limits through within-model interventions, closely matched release variants, graded controls, explicit tensor locations, and conservative causal language. Paper-series theory, embodied extensions, local learning, and detailed goal-latent architectures remain in Paper 0 and the roadmap.

11 Conclusion

Paper 1 asks a bounded causal question: what does Transformer residual depth do under counterfactual task context, and do learned gates select updates with independently measurable consequences? The answer requires candidate/effective update pairs, causal block utility, later repair, matched controls, and replication. Gating is evidence only to the extent that interventions and downstream effects support it. The architecture remains subordinate to the result: refinement, complementarity, conditional computation, interference, and null outcomes are all admissible conclusions.

References

[1] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of Neural Network Representations Revisited. *ICML*, 2019.

[2] A. Graves. Adaptive Computation Time for Recurrent Neural Networks. *arXiv:1603.08983*, 2016.

[3] D. Raposo et al. Mixture-of-Depths: Dynamically Allocating Compute in Transformer-Based Language Models. *arXiv:2404.02258*, 2024.

[4] M. Elbayad et al. LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. *arXiv:2404.16710*, 2024.

[5] Z. Qiu et al. Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free. *arXiv:2505.06708*, 2025.